

Consistency-Guided xApp Coordination in the Near-RT RIC: Incremental Propagation, Explanation-Guided Exact Repair, and Independently Verified Decisions

Lillia Ouali^{1*}, Madani Bezoui², Kamal Amroun¹, Ahcene Bounceur³

^{1*} Department of Computer Science, University of Bejaia, Bejaia, Algeria .

² LINEACT, CESI, Nancy, France .

³ University of Sharjah, Sharjah, United Arab Emirates .

*Corresponding author(s). E-mail(s): lillia.ouali@univ-bejaia.dz;

Contributing authors: mbezoui@cesi.fr; kamal.amroun@univ-bejaia.dz;
abounceur@sharjah.ac.ae;

Abstract

Independently developed xApps issue concurrent control requests through the near-real-time RAN Intelligent Controller (RIC) of an Open RAN, and these requests may be mutually incompatible or jointly violate service-level agreements. We model multi-xApp arbitration as a dynamic finite-domain constraint optimization problem and propose CONSISTXAPP, a coordination pipeline that maintains generalized arc consistency incrementally across decision epochs, repairs invalid candidate decisions through an exact local subproblem whose scope is grown by propagation explanations and solver conflict cores, and forwards an action only after an independent verifier has checked every registered hard constraint. The verifier is itself validated by exhaustive enumeration against an independent oracle, a second implementation sharing no code, property-based testing, and oracle-labelled input-mutation testing, with zero divergence across 680,450 checks. On five O-RAN conflict scenarios, CONSISTXAPP attains 100% verified feasibility with a 1.8 ms median latency, faster than complete CP-SAT (2.8 ms, paired Wilcoxon $p < 10^{-33}$), and every repair carries an auditable certificate replayable against the registered model. On the 248 repairs triggered by the operational pipeline, expanding repair cores succeed where fixed cores fail on 8.5% of cases (exact McNemar $p < 10^{-5}$) while remaining approximately half the size of full-scope solving. The decisive advantage is amortized: when 2% of the conflict relations change between epochs, repairing the previous verified decision costs a median of 0.05–4.9 ms per epoch for 100–1,000 variables, against 2.8–25.6 ms for re-solving from scratch with or without solution hints, and the advantage holds in every one of ten paired episodes per size, tail latencies, however, reach re-solve parity at the 95th percentile and cross over beyond it. We report the boundaries of the approach with equal weight: cold-start coordination of isolated instances offers no advantage over a modern solver, local repair pays a quantified price in objective quality relative to exact optimization (10.8 points of kept-request rate on average), and a continuous-relaxation stage that we evaluated degrades both latency and feasibility and is excluded from the pipeline.

Keywords: O-RAN, xApp conflict management, dynamic constraint satisfaction, generalized arc consistency, explanation-based repair, constraint programming, near-real-time RIC

1 Introduction

The Open Radio Access Network (O-RAN) architecture disaggregates the radio access network and exposes its control through programmable components. Its near-real-time RAN Intelligent Controller (RIC) hosts xApps - third-party applications that subscribe to measurements and issue control requests over the E2 interface within control loops of tens of milliseconds to one second [1]. Because xApps are developed independently, often by different vendors, they may simultaneously write the same parameter, steer different parameters toward incompatible operating points, or jointly violate a service-level agreement (SLA). The O-RAN Alliance therefore assigns a conflict-mitigation function to the near-real-time RIC [2], and a growing literature classifies xApp conflicts as direct, indirect, or implicit and proposes detection and resolution mechanisms [3, 4].

Most existing mechanisms answer the question of *what* a candidate action will do to the network, through profiling, digital twins, learning, or bargaining. This paper addresses the complementary question that arises once candidate actions and conflict relations have been registered: how should a controller reason over finite action domains, under a near-real-time budget, so that only decisions satisfying every registered hard constraint reach the network - and so that each intervention can be audited afterwards?

We answer with CONSISTXAPP, a coordination pipeline built from three mechanisms of constraint programming that, to our knowledge, have not been combined for O-RAN control. First, an *incremental generalized arc-consistency* (GAC) stage removes candidate actions that have no support in any incident hard relation, reusing propagation state across decision epochs and restoring values soundly when a relation is relaxed. Second, when a decoded candidate violates some relations, an *explanation-guided exact repair* solves a small induced subproblem: the violated constraints seed a variable core, the remaining variables are fixed to their decoded values, and the core grows only as directed by propagation explanations or by the solver’s own infeasibility cores, escalating to the full model only if needed. Third, an *independent verifier*, validated against an exhaustive oracle and a second implementation, gates every forwarded

action. The pipeline is deliberately conservative: it never trusts its own optimizer.

A modern constraint solver is a demanding yardstick for such a design. OR-Tools CP-SAT solves each of our scenario instances from scratch in under 3 ms, so no coordination layer can justify itself on isolated instances of that size, we report this honestly rather than hide it. The case for CONSISTXAPP is *amortized*. Near-real-time coordination is a sequence of closely related problems: between two epochs, only a fraction of the xApps, domains, and context-dependent relations changes. Re-solving from scratch pays the full model cost at every epoch - and, as our experiments show, warm-starting the solver with the previous solution does not reduce that cost, whose measured components (model construction plus full-model search) both scale with the model rather than with the change. Repairing the previous verified decision instead pays a cost proportional to the number of violated relations. At one thousand variables with 2% of relations changing per epoch, verified local repair is five times faster in the median than either evaluated re-solve strategy, and the gap widens with model size.

The contributions of this paper are as follows. We formulate multi-xApp coordination as a dynamic finite-domain constraint optimization problem with context-dependent hard relations and a temporal-stability objective. We design an incremental GAC layer with sound value restoration after relaxations, prove that its filtering preserves every globally feasible assignment, and show by a symmetric counterexample why filtering alone cannot guarantee valid decoding. We introduce explanation-guided exact repair with monotone core expansion and conditional completeness, on the repairs triggered by the operational pipeline, expanding cores succeed significantly more often than fixed cores and remain approximately half the size of full-scope solving at a lower cost. We validate the independent verifier through four gates - exhaustive oracle, second implementation, property-based testing, oracle-labelled input mutation - which, to our knowledge, have not previously been reported for O-RAN conflict management. Finally, we characterize the operational regime of the architecture with paired, multiplicity-corrected statistics at the episode level: no cold-start advantage at up

to 1,000 variables, a growing amortized median advantage in sequential operation with honestly reported tail behaviour, a quantified objective-quality gap relative to exact optimization, and a negative result for a continuous-relaxation stage that we evaluated and excluded.

Section 2 reviews related work. Section 3 defines the coordination model. Section 4 presents CONSISTXAPP and Section 5 its properties. Sections 6 and 7 describe the experimental setup and results. Section 8 discusses limitations and Section 9 concludes.

2 Background and Related Work

The near-real-time RIC supports control loops between tens of milliseconds and one second, xApps subscribe to key performance measurements (KPMs), process network context, and issue E2 control requests [1]. Experimental frameworks such as CoO-RAN, OpenRAN Gym, and FlexRIC support the development and closed-loop evaluation of such controllers [5–7]. Three conflict classes are commonly distinguished [3]. A *direct* conflict arises when two xApps request incompatible values of the same parameter, for example a coverage xApp raising transmission power that an energy-saving xApp is lowering. An *indirect* conflict arises when different controlled parameters influence a common KPM, as when cell-individual offsets and handover thresholds jointly determine user association. An *implicit* conflict couples distinct parameters and KPMs through network dynamics, as when sleeping a cell degrades mobility robustness. Direct conflicts are checkable syntactically, indirect and implicit conflicts require a registered parameter–KPM dependency model, a profile, or a conservative operator rule.

Proposed management mechanisms span a conflict-mitigation framework embedded in the RIC [3], QoS-aware optimization that maximizes the number of xApps whose requirements are met [4, 8], cooperative bargaining over conflicting parameters [9], pre-deployment profiling with statistical models and hierarchical graphs [10], digital-twin-based impact evaluation [11], and rule-based mitigation in mobility-versus-energy case studies [12]. Closest to our auditability goal,

XAI4C applies explainable-AI techniques to conflict detection and mitigation [13], its explanations are post-hoc attributions of a learned detector, whereas the explanations produced here are logical premises replayable against the registered constraint model and checkable by an independent program.

On the constraint-programming side, propagation removes domain values that cannot participate in any locally supported tuple, for non-binary relations this is generalized arc consistency, efficiently enforceable for positive table constraints by simple tabular reduction and its successors [14, 15]. Local consistency is not a decision procedure - a GAC network may admit no solution - but it detects some contradictions without search and shrinks the space handed to an exact solver. Dynamic constraint satisfaction studies sequences of related problems and incremental consistency maintenance [16], and explanations that justify value removals or contradictions support diagnosis and repair [17]. Modern solvers such as OR-Tools CP-SAT combine presolve, propagation, clause learning, and optimization [18, 19], which makes them both a component of our repair stage and the baseline any coordination layer must face. The contribution of this work is the integration of incremental propagation, explanation-guided local repair, and independently validated verification into a budget-aware xApp coordination loop, together with an evaluation that identifies the regime - sequential, not cold-start - in which the integration pays.

3 System Model

3.1 Epochs, actions, and context-dependent relations

The near-real-time RIC operates over decision epochs $t = 1, \dots, T$. At epoch t , each active xApp action variable $i \in \mathcal{A}_t$ has a finite base domain $D_i^t = \{a_{i,1}^t, \dots, a_{i,d_i}^t\}$ whose values encode concrete controls: accepting or rejecting a request, selecting a target cell or an execution slot, choosing a discrete power level, a handover offset, or a resource-allocation profile, or executing a registered safe no-operation. The discrete variable $x_i^t \in D_i^t$ is the coordinated action selected for variable i , the time index is omitted when a single epoch is considered.

A constraint $c \in \mathcal{C}_t$ has scope $S_c \subseteq \mathcal{A}_t$ and an allowed relation

$$R_c(\mathbf{s}_t) \subseteq \prod_{i \in S_c} D_i^t, \quad (1)$$

where $\mathbf{s}_t$ is the observed and registered network context. An assignment $\mathbf{x}^t$ satisfies c if $\mathbf{x}_{S_c}^t \in R_c(\mathbf{s}_t)$. Relations may be binary or higher-order and are generated from operator policies, parameter ownership, SLA and QoS thresholds, validated xApp profiles, a calibrated simulator or digital twin, or conservative safety rules. Hard constraints must hold before a decision executes, soft constraints only contribute to utility. The hard model at epoch t is $\mathcal{M}_t = (\mathcal{A}_t, \mathcal{D}_t, \mathcal{C}_t, \mathbf{s}_t)$.

Between epochs, xApps activate or deactivate, candidate actions and constraints appear or disappear, relations tighten or relax, and SLA thresholds move. The modified variables and constraints form the change set Δ_t . Restrictive and relaxing updates must be distinguished: after a restriction, previous support information remains reusable, whereas after a relaxation a previously removed value may have regained support and must be restored from its base domain before propagation is re-established (Proposition 4).

3.2 Objective

Let $u_i(a_i, \mathbf{s}_t)$ be the utility of selecting action a_i for variable i , weighted by $\omega_i \geq 0$, and let $v_q \geq 0$ be soft-violation penalties with weights λ_q . The static utility is

$$U(\mathbf{x}, \mathbf{s}_t) = \sum_{i \in \mathcal{A}_t} \omega_i u_i(x_i, \mathbf{s}_t) - \sum_{q \in \mathcal{Q}_t} \lambda_q v_q(\mathbf{x}, \mathbf{s}_t). \quad (2)$$

With $\mathbf{x}^{t-1}$ the previous verified decision and $\kappa_i \geq 0$ reconfiguration costs, the weighted action churn is

$$H(\mathbf{x}^t, \mathbf{x}^{t-1}) = \sum_{i \in \mathcal{A}_t \cap \mathcal{A}_{t-1}} \kappa_i \mathbf{1}[x_i^t \neq x_i^{t-1}], \quad (3)$$

and the coordination problem at epoch t is

$$\max_{\mathbf{x}^t} U(\mathbf{x}^t, \mathbf{s}_t) - \mu H(\mathbf{x}^t, \mathbf{x}^{t-1}) \quad (4)$$

$$\begin{aligned} \text{s.t. } & x_i^t \in D_i^t, \quad i \in \mathcal{A}_t, \\ & \mathbf{x}_{S_c}^t \in R_c(\mathbf{s}_t), \quad c \in \mathcal{C}_{t, \text{hard}}, \end{aligned} \quad (5)$$

where $\mu \geq 0$ trades instantaneous utility against temporal stability. Dwell-time and cooldown rules are hard relations when their violation is not permitted.

3.3 Worked example

Consider one cell with a coverage xApp controlling power $x_1 \in \{\text{low}, \text{med}, \text{high}\}$, an energy-saving xApp controlling the cell state $x_2 \in \{\text{sleep}, \text{on}\}$, and two SLAs (minimum edge throughput, maximum handover-failure rate). The registered relation $R_{12} = \{(\text{med}, \text{on}), (\text{high}, \text{on}), (\text{low}, \text{sleep})\}$ forbids raising coverage from a sleeping cell and forbids low power on an active cell. Suppose the xApps request $x_1 = \text{high}$ and $x_2 = \text{sleep}$, requests are soft preferences that enter decoding through utility, while the base domains remain multivalued (a hard singleton restriction would instead be detected as a wipeout by propagation). GAC removes nothing: high is supported by (high, on) and sleep by (low, sleep), yet the jointly decoded request violates R_{12} . The verifier rejects it, the violated relation seeds the repair core $\{x_1, x_2\}$, and exact repair returns a verified alternative such as (high, on) - or (low, sleep) if the energy SLA dominates. Propagation prunes unsupported actions, repair resolves incompatible combinations of individually supported actions.

4 The ConsistXApp Framework

Figure 1 positions CONSISTXAPP in the near-real-time RIC control path: candidate xApp actions are filtered by incremental GAC, decoded, repaired when necessary, and independently verified before any E2 message is emitted.

4.1 Incremental consistency filtering

The propagation stage maintains, next to each base domain D_i , a filtered domain $D_i^* \subseteq D_i$, the separation matters because a later relaxation may restore support for a removed value. The support set of value $a \in D_i$ in constraint c is

$$\text{Supp}_c(i, a; \mathbf{D}) = \{z \in R_c(\mathbf{s}_t) : z_i = a \wedge z_j \in D_j, \forall j \in S_c\}, \quad (6)$$

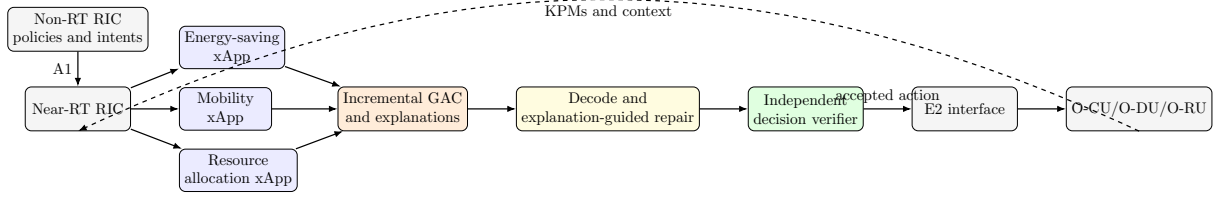


Fig. 1 CONSISTXAPP in the near-real-time RIC control path. Incremental generalized arc consistency filters unsupported actions, propagation explanations guide exact local repair, and every final action is independently verified before reaching E2.

and a hard network is *generalized arc consistent* when every value of every variable has a non-empty support in every incident relation. The revision operator

$$\text{Revise}_{c,i}(\mathbf{D}) : D_i \leftarrow \{a \in D_i : \text{Supp}_c(i, a; \mathbf{D}) \neq \emptyset\} \quad (7)$$

is applied through a propagation queue until a fixed point $\mathbf{D}^* = \text{GAC}(\mathbf{D}, \mathcal{C}_{\text{hard}}, \mathbf{s}_t)$ is reached. An empty filtered domain (a *wipeout*) proves that no feasible assignment extends the current domains, the converse does not hold.

Across epochs, the propagator works incrementally (Algorithm 1). The queue is seeded only with the constraints affected by Δ_t and those sharing an affected variable. For a purely restrictive transition, cached supports and previous deletions are reused as long as their premises remain valid. For a transition containing a relaxation, the affected connected component is first restored to its base domains and current allowed tuples, then consistency is re-established, this restoration rule prevents stale deletions from being carried unsoundly across a relaxed context. For explicit positive tables the implementation records the active tuples, the last known support of each variable–value pair, the context identifier under which that support was valid, and the explanation of each removal, these caches accelerate propagation without changing the fixed-point semantics.

4.2 Propagation explanations

When value a is removed from D_i by constraint c , the propagator records an explanation $\mathcal{E}(i, a) \subseteq \mathcal{C}_{\text{hard}} \cup \mathcal{L}$, where $\mathcal{L}$ collects temporary domain restrictions and boundary decisions. The explanation contains premises sufficient to establish that

every tuple of R_c assigning a to x_i is incompatible with the current domains: for each candidate support tuple, at least one premise states why that tuple is inactive. A wipeout explanation for variable i is the union $\mathcal{E}_\emptyset(i) = \bigcup_a \mathcal{E}(i, a)$ over its removed base values. Explanations are not required to be cardinality-minimal, they are required to be logically valid relative to the registered model, reproducible from the propagation trace, bound to a context identifier, and accepted by a separately implemented explanation checker that replays the premises and verifies that every supporting tuple is disabled.

4.3 Decoding and explanation-guided exact repair

If propagation ends without wipeout, a deterministic decode $\hat{x}_i \in D_i^*$ selects, per variable, the maximum-utility (or documented tie-broken) value of the filtered domain. Local support does not imply joint validity (Section 5), so the decoded candidate is tested and, if it violates hard relations, repaired exactly (Algorithm 2).

The violated relations $\mathcal{C}^0 = \{c : \hat{x}_{S_c} \notin R_c(\mathbf{s}_t)\}$ seed the variable core $\mathcal{A}^0 = \bigcup_{c \in \mathcal{C}^0} S_c$. At iteration r , variables outside the core are fixed to their decoded values and variables inside retain their filtered domains, GAC is enforced on this induced model and the subproblem is passed to CP-SAT. The expansion order is deterministic. If the induced propagation wipes out, the wipeout explanation supplies the constraints and variables to add:

$$\mathcal{A}^{r+1} = \mathcal{A}^r \cup \text{VarLit}(\mathcal{E}_\emptyset^r) \cup \bigcup_{c \in \mathcal{C}^{r+1}} S_c, \quad (8)$$

where $\mathcal{C}^{r+1} = \mathcal{E}_\emptyset^r \cap \mathcal{C}_{\text{hard}}$ are the hard constraints named by the wipeout explanation and VarLit returns the variables mentioned by the explanation literals. If propagation succeeds but the local solve is infeasible, the assumption-core variant releases exactly the boundary variables identified by the solver: each boundary fixing is expressed as a reified Boolean literal $b_j \Rightarrow (x_j = \hat{x}_j)$ registered through `AddAssumptions`, and on infeasibility the literal indices returned by `SufficientAssumptionsForInfeasibility` are mapped back to variables (an empty returned core conservatively releases all fixed boundary variables, UNKNOWN is treated as a timeout). The core-extraction model contains no objective. When neither an explanation nor a solver core yields a new variable - including in the default explanation variant, whose local solve fixes the boundary with plain equality constraints - a deterministic frontier rule adds the constraints adjacent to the current core. The three mechanisms are logged and distinguished in the evaluation. Repair terminates with a verified decision, with escalation to the full variable set (where it coincides with complete solving), at the deadline, or with a registered safe fallback.

Each repair emits a *repair certificate*: the decoded candidate, the violation witnesses (the decoded tuples of the violated relations), the core trajectory with the expansion source of every step (wipeout explanation, solver assumption core, or frontier), the released boundary variables, the final assignment, and the verifier outcome. Propagation explanations and solver cores are components of this certificate when expansions occur, the certificate itself exists for every repair and is replayable by the independent checker. Replaying the record establishes that the decoded candidate violated the listed relations, that each expansion was justified by its recorded source, and that the final assignment satisfies every registered hard constraint, it is an auditable trace and does not by itself certify core minimality or objective optimality.

The repair objective is lexicographic: satisfy every hard relation, minimize weighted reconfiguration against the previous verified decision (Eq. (3)), minimize deviation from the decode, maximize the static utility of Eq. (2). Stability is enforced once, at the second level, so the last level deliberately uses the static rather than the churn-penalized utility.

A safe fallback must be an explicitly registered action of the current model - rejecting the conflicting requests, holding the previous configuration, or a safe no-operation - and must pass the same verifier as an optimized decision, in particular, the previous configuration is re-verified under the current context before reuse.

4.4 Independent verification and its validation

The verifier is implemented separately from propagation, explanation construction, and repair. It receives the context identifier, the selected actions, the base domains, the hard relations, and the SLA thresholds, and accepts exactly the set

$$\mathcal{F}(\mathbf{s}_t) = \{\mathbf{x} : x_i \in D_i, \mathbf{x}_{S_c} \in R_c(\mathbf{s}_t) \forall c \in \mathcal{C}_{\text{hard}}\}. \quad (9)$$

Proposition 1 (Decision soundness). *If CONSISTXAPP forwards an action $\mathbf{x}$ to the E2 interface, then $\mathbf{x} \in \mathcal{F}(\mathbf{s}_t)$ relative to the registered domains, relations, context, and verifier semantics.*

Proof A candidate is forwarded only if the verifier accepts it, and the verifier checks domain membership and every registered hard relation directly. $\square$

The proposition itself is an architectural invariant - it holds by construction of the forwarding rule - so its practical force rests entirely on the verifier being correct. The verifier is therefore validated by four gates, executed before any campaign and reported with the results. Gate G1 enumerates every assignment of randomly generated small instances and compares the verifier with an independently coded oracle. Gate G2 rebuilds a declarative CP-SAT model of the instance from a JSON-normalized form through a separate parser and requires triple agreement. Gate G3 applies property-based testing - determinism, invariance under constraint permutation and variable renaming, and explicit rejection of out-of-domain values, missing variables, and stale contexts - to randomized inputs. Gate G4 performs oracle-labelled input mutation (a fuzzing-style test, not code-mutation testing): mutants of planted-feasible solutions are labelled by the oracle and the verifier must classify every mutant identically, a mutant

Algorithm 1 Incremental GAC with restoration after relaxation

Input: warm domains D_{t-1}^* , active tables, change set Δ_t

Output: filtered domains D_t^* or WIPEOUT

- 1: classify Δ_t as restrictive, relaxed, mixed, or utility-only
 - 2: **if** Δ_t contains a relaxation **then** restore the affected constraint-graph component to its base domains and current allowed tuples
 - 3: apply restrictive deletions directly to D_{t-1}^*
 - 4: seed the worklist with constraints incident to changed variables and relations
 - 5: **while** worklist non-empty: pop c , revise every $i \in S_c$, if some $D_i^* = \emptyset$ **return** WIPEOUT, re-queue constraints incident to a shrunk domain
 - 6: **return** D_t^*
-

Algorithm 2 Explanation-guided repair-core expansion

Input: decoded $\hat{x}$, hard model, deadline

Output: verified repaired decision (local or full-scope), or a registered safe FALLBACK

- 1: $\mathcal{C}^0 \leftarrow$ violated relations, $\mathcal{A}^0 \leftarrow \bigcup_{c \in \mathcal{C}^0} S_c$
 - 2: **repeat**:
 - 3: fix boundary variables $j \notin \mathcal{A}^r$ to $\hat{x}_j$, enforce GAC on the induced model
 - 4: solve the induced subproblem with CP-SAT (boundary fixings as assumptions)
 - 5: **if** solved and the verifier accepts the full assignment **then return** it
 - 6: **if** $\mathcal{A}^r = \mathcal{A}$ (the last solve was the complete model) or the deadline has passed **then return** FALLBACK
 - 7: expand $\mathcal{A}^{r+1}$ via the assumption core, the wipe-out explanation, or the frontier rule (Eq. (8))
-

that remains feasible counting as feasible. The oracle and the second verifier share no relation parser, tuple normalization, domain indexing, or context-validation code with the production verifier, the JSON normalization consumed by Gate G2 is produced by a separate serializer, and malformed inputs are covered by the rejection properties of Gate G3. A single divergence at any gate blocks the campaign.

5 Analysis

Proposition 2 (Solution preservation). *For every globally feasible assignment $\mathbf{x} \in \mathcal{F}(\mathbf{s}_t)$ and every variable i , $x_i \in D_i^*$: GAC filtering removes no value used by a feasible assignment.*

Proof A value is removed only when it has no support under the current domains. If $\mathbf{x}$ is feasible and $x_i = a$, the tuple $\mathbf{x}_{S_c} \in R_c$ supports a as long as its components remain in their domains. Initially all components are present, by induction over the revision sequence, no component of $\mathbf{x}$ can be the first one removed, since the others would still support it. $\square$

Filtering alone does not guarantee valid decoding. Consider two requests that must occupy different slots: $D_1 = D_2 = \{0, 1\}$ with $R_{\neq} = \{(0, 1), (1, 0)\}$. Every value of every variable has a support, so the network is GAC and nothing is removed, yet any deterministic decode that breaks the two symmetric ties identically - the natural behaviour of a per-variable rule on identical domains - returns $(0, 0)$ or $(1, 1)$ and violates $R_{\neq}$. Propagation and repair therefore address different failure modes: propagation removes individually unsupported actions, repair resolves incompatible combinations of individually supported ones. This is why CONSISTXAPP treats decoding as a proposal and never as a decision.

Proposition 3 (Monotone expansion and conditional completeness). *The repair cores satisfy $\mathcal{A}^0 \subseteq \mathcal{A}^1 \subseteq \dots \subseteq \mathcal{A}$, and if every unsuccessful non-full iteration adds at least one new variable, the full set is reached after at most $|\mathcal{A}| - |\mathcal{A}^0|$ expansions. If the core reaches $\mathcal{A}$, the exact solver is complete, its time limit is not hit, and every registered finite-domain relation is encoded exactly in the solver model, then repair returns a feasible assignment iff the registered hard model is feasible.*

Proof Monotonicity is by construction of Eq. (8), and the expansion bound follows by counting. With a full core no variable is fixed to a boundary value, so the repair model coincides with the complete hard model and the claim follows from solver completeness. $\square$

A repair restricted to a proper subset is *not* complete: its infeasibility may be caused by the fixed boundary even when the model is feasible,

which is precisely the information carried by the assumption core, and why local infeasibility is never reported as model infeasibility.

Proposition 4 (Necessity of restoration after relaxation). *If value a was removed at epoch $t - 1$ for lack of support in $R_c(\mathbf{s}_{t-1})$ and the relation is relaxed at epoch t , keeping the deletion without re-evaluation may exclude a feasible assignment of $\mathcal{M}_t$.*

Proof The relaxation may introduce a tuple $\mathbf{z} \in R_c(\mathbf{s}_t)$ with $z_i = a$ whose other components are compatible with the current domains, $\mathbf{z}$ is then a new support for a and the previous no-support explanation is no longer valid. $\square$

One complete scan of the positive tables costs $O(\sum_c |R_c| |S_c|)$, actual propagation cost depends on support caching, the number of removals, and arity, while exact repair remains exponential in the worst case. None of these constants is assumed favourable: the evaluation reports propagation time, explanation sizes, core sizes and expansions, solver times, end-to-end latency, and deadline compliance, and it treats complete CP-SAT-which embeds its own presolve, propagation, and learning - as the baseline that the architecture must justify itself against.

6 Experimental Setup

6.1 Platform, scenarios, and instance suites

The evaluation uses a Python-based O-RAN coordination simulator comprising the instance generators, the observable GAC propagator with its explanation recorder and checker, the OR-Tools CP-SAT adapters, the repair engine, the independent verifier, and a transparent pseudo-physical network model that maps coordinated actions (power, resource-block fraction, handover offset, sleep state) to SINR, throughput, energy per bit, Jain fairness, and a handover-failure indicator over N_c cells and N_u uniformly placed users with distance-based path loss. The model is deliberately simple: its role is to produce reproducible KPMs, not to replace a system-level simulator.

Allowed relations encode KPM thresholds, ownership, and sleep/wake exclusivity, and every generated hard relation retains a planted feasible assignment so that coordination failure is distinguishable from model infeasibility. A separate diagnostic suite of controlled infeasible instances exercises wipeout detection and explanation validity.

Table 1 lists the five conflict scenarios, they use 12–30 action variables with domains of three or four values, and 50 seeds each (250 matched instances). The dynamic suite runs the same scenarios over 30 episodes of 100 epochs (15,000 decision epochs) with a fixed schedule of restrictive, relaxed, mixed, and utility-only transitions modifying 1–50% of the model. Mixed transitions, which simultaneously tighten some relations and relax others, make up 84% of the schedule because that is the typical per-epoch behaviour, the purely restrictive, purely relaxed, and utility-only epochs are deliberately isolated corner cases that let each propagation mechanism be measured separately (Table 9). A *repair-stress* suite provides harder repair cases: 50 planted-feasible instances of 24 variables ($d = 4$, 34 relations, 70% equality-type constraints) whose candidate is the planted solution with three variables flipped, so that the initial violated scope couples several relations at once. Two further suites extend the size axis. The *cold-start* suite contains near-threshold planted-feasible instances with $n \in \{30, 100, 300, 600, 1000\}$ variables ($d = 6$, $|\mathcal{C}| = 2n$, tightness 0.25, twenty seeds per size), solved once from scratch. The *sequential* suite runs ten independent episodes of thirty epochs per size and change rate: between epochs a fraction $\delta \in \{2\%, 10\%\}$ of the relations is re-randomized while a planted assignment is preserved, so every epoch remains feasible by construction. Because a static planted anchor could make the problem artificially easy, a *drift* variant additionally re-plants 1% of the variables per epoch, so the feasibility anchor itself moves through the solution space, the fraction of epochs on which the previous decision remains feasible is reported for every configuration.

Table 1 O-RAN coordination scenarios.

Scenario	xApps	Controlled parameters	Principal KPMs	Conflict mechanism
S1: direct power conflict	Coverage and energy-saving	Transmission power, cell state	Coverage, energy, SINR,	Incompatible requests on the same parameter
S2: radio-resource conflict	Power-control and resource-allocation	Power level, resource block profile	Throughput, interference, loss	Different parameters, common KPMs
S3: mobility-energy conflict	Mobility-robustness and energy-saving	Handover offsets, sleep/wake	Handover failures, ping-pong, energy	Sleeping modifies coverage and mobility
S4: load-balancing conflict	Load-balancing and mobility	Cell offsets, handover thresholds	Cell load, edge throughput, handovers	Joint control of user association
S5: multi-xApp stress	All xApps	Joint action space	All registered KPMs and SLAs	Mixed direct, indirect, implicit

6.2 Methods, metrics, and statistical protocol

All methods share the input model, timing harness, and verifier, and every method’s candidate-generation rule is stated explicitly, since methods with different decoders are not comparable as repair ablations. CONSISTXAPP and the solver baselines use the deterministic *utility decode*: per variable, the documented tie-broken best value of the GAC-filtered domain. The comparison set contains *uncoordinated* forwarding and *static priority* as execution-semantics references, *GAC + utility decode* (the pipeline candidate with repair disabled) and *GAC + continuous decode* (a message-passing-style relaxation decode, no repair) as ablations, together with the full *continuous-relaxation pipeline*, complete CP-SAT and *GAC + CP-SAT* as the solver baselines, CONSISTXAPP itself (GAC, utility decode, explanation-guided repair, verification), plus two variants whose repair core grows by fixed graph radius or by solver assumption cores, and two *objective-inspired* baselines run on identical instances - a QoS-aware optimizing baseline that reproduces the kept-request objective of QACM [4] without its protocol machinery, and a bargaining baseline maximizing a discretized Nash-social-welfare surrogate as in cooperative-game mitigation [9] - both solved exactly by CP-SAT and gated by the same verifier, they are objective reproductions, not reimplementations of the published systems. To test

whether explanation-guided repair remains valuable when initialized from a competent candidate generator rather than the deliberately weak utility decode, we additionally evaluate three inexpensive *conflict-aware* decoders on the static suite: a *greedy* sequential assignment with immediate constraint checks, a *forward-checking* sequential search with chronological backtracking under a 50 ms budget, and a *min-conflicts* local search seeded from the utility decode; each emits a candidate that is either accepted directly by the verifier or completed by the same explanation-guided repair. In the sequential suite the per-epoch candidate of incremental repair is the *previous verified decision*, re-verified under the updated model and repaired only when violated, its competitors are full CP-SAT re-solve and CP-SAT re-solve warm-started with the previous solution as a hint, both instrumented to report model-construction and search time separately. An objective-quality experiment completes the picture: on every scenario instance, the kept-request rate of explanation-guided repair of the preferred-request vector is compared with the exact optimum of that objective computed by CP-SAT.

Table 2 fixes the outcome terminology. Verified feasibility counts decisions accepted by the verifier, a verified fallback is never silently counted as an optimized decision. The 100 ms control budget is a *post-hoc compliance threshold* checked at stage boundaries (after propagation, decoding,

repair iterations, and verification), not a preemptive interrupt: a stage that overruns is completed and its full latency reported, which is why non-compliant methods can record latencies well above 100 ms. Filtering is quantified by the value- and tuple-reduction ratios $r_D = 1 - \sum_i |D_i^*| / \sum_i |D_i|$ and $r_R = 1 - \sum_c |R_c^*| / \sum_c |R_c|$, where R_c^* is the active part of R_c after propagation. Every stochastic configuration uses 50 seeds (dynamic episodes: 30, cold-start: 20 seeds per size, sequential: 10 episodes per size and change rate), with paired methods on identical instances and traces. Binary outcomes are reported as integer counts with exact McNemar tests on discordant pairs (complete-case analysis, excluded errored instances are reported explicitly), continuous metrics as medians, tail percentiles, paired Wilcoxon signed-rank tests, matched-pairs rank-biserial correlations, and Hodges–Lehmann estimates of the median paired difference with bootstrap intervals, Holm’s procedure corrects each comparison family. In the sequential suite the *episode*, not the epoch, is the experimental unit: per-episode median latencies are compared across methods, since epochs within an episode are temporally dependent. Timeouts are right-censored, never converted to latencies. The scenario, dynamic, and temporal campaigns ran on the reference host of the primary manifest (28-core Intel host, Windows 11, Python 3.13, OR-Tools 9.14), the ablation-on-operational-candidates, cold-start, sequential, objective-gap, baseline, and verifier-validation campaigns ran in a documented secondary environment (2-vCPU Linux container, Python 3.10, OR-Tools 9.15), seeds, versions, and command lines are recorded in the two manifests shipped with the artifact, and all paired comparisons are within-environment.

7 Results

7.1 Trust layer

All four verifier-validation gates pass with zero divergence (Table 3): 60,650 exhaustively enumerated assignments against the independent oracle, 1,800 triple-agreement checks with the declarative CP-SAT re-implementation, 602,000 property-based checks, and 16,000 oracle-labelled input mutants (13,594 invalid) classified identically -

Table 2 Decision-outcome terminology.

Term	Meaning
Direct decode	decode accepted by the verifier without repair
Local repair	invalid decode corrected on a proper variable subset
Full-scope repair	repair after expansion to the complete model
Fallback	registered safe fallback accepted by the verifier
Blocked	no verifier-accepted action produced
Verified feasibility	direct + local + full + verified fallback
Deadline compliance	verified decisions completed within the budget

Table 3 Verifier-validation gates (zero divergence required).

Gate	Content	Checks	Div.
G1	exhaustive oracle, 500 instances	60,650	0
G2	triple agreement, CP-SAT re-impl.	1,800	0
G3	property-based testing	602,000	0
G4	labelled input mutants (13,594 invalid)	16,000	0

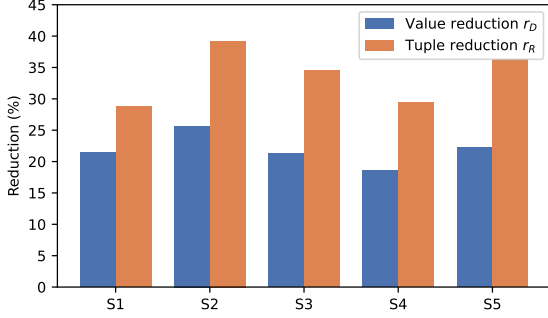
680,450 checks in total. The propagation sanity gates also pass: on instances small enough for exhaustive enumeration no removed value belonged to a feasible assignment, incremental propagation reproduced the recomputed domains on all 15,000 dynamic epochs, and the explanation checker accepted all valid and rejected all corrupted explanations. These checks make the soundness guarantee of Proposition 1 operational, they do not validate the registered model against a physical RAN.

7.2 Consistency filtering

Propagation removes 21.7% of candidate values and 34.1% of relation tuples pooled over the static suite (Table 4, Fig. 2), with sub-millisecond median GAC times. The reduction is largest in S2 and smallest in S4, and the per-scenario spread shows why aggregate averages alone would mislead: instances on which GAC filters little coexist with instances in which a third of the action space disappears before any search.

Table 4 Consistency-propagation results (50 epochs per scenario).

Scenario	Val before	Val after	r_D (%)	Tup before	Tup after	r_R (%)	Med GAC (ms)	P95 GAC (ms)
S1	1800	1414	21.4	2403	1713	28.7	0.07	0.09
S2	3600	2676	25.7	6083	3697	39.2	0.15	0.18
S3	4000	3148	21.3	7419	4858	34.5	0.17	0.20
S4	4800	3908	18.6	9694	6846	29.4	0.20	0.24
S5	6000	4662	22.3	11747	7488	36.3	0.26	0.31

**Fig. 2** Domain and relation-table reduction obtained by generalized arc consistency.

7.3 Verified feasibility under the control budget

Table 5 reports the static-suite outcomes as integer counts, and Table 6 makes the key comparisons inferential. The utility decode alone almost never produces a directly valid joint decision - 2 direct decodes out of 250, the remaining 248 blocked without repair - which isolates the contribution of repair exactly: CONSISTXAPP converts the same 248 candidates into verified local repairs and reaches 100% verified feasibility with full deadline compliance, alongside complete CP-SAT (median 2.8 ms) and GAC + CP-SAT (2.6 ms). CONSISTXAPP is the fastest of the three (1.8 ms, paired Wilcoxon $p < 10^{-33}$, matched-pairs rank-biserial $r = -0.89$, Hodges-Lehmann median difference -0.85 ms, 95% CI $[-0.93, -0.72]$), a statistically unambiguous but operationally modest one-millisecond margin at this instance size, the substantive advantages at this scale are the audit trail - every repair carries a replayable certificate - and, at larger scale, the amortized behaviour of Section 7.7. The objective-inspired baselines also reach 100% verified feasibility (median 2.2 and 3.5 ms in the secondary environment) while keeping on average 64% of the xApp-preferred

actions, they optimize a different objective and offer no per-decision audit trail. Uncoordinated forwarding and static priority produce no verifier-accepted action on this suite: under near-threshold planted relations, a random or greedy per-variable proposal essentially never satisfies every hard relation jointly, so both serve as execution-semantics references only.

The lower block of Table 5 documents a design alternative that we evaluated and rejected: a continuous-relaxation stage, inspired by message passing on the filtered relation tables [20], inserted between propagation and repair. Whatever its diagnostic appeal, it is operationally counterproductive - median latency rises to about 80 ms, deadline compliance falls to 57%, and 6–7 instances block outright - and the choice of repair-core rule cannot rescue it. CONSISTXAPP therefore ships without it, and the finding is reported as a negative result rather than omitted.

7.4 Repair-strategy ablation

Table 7 compares core-construction strategies on the 248 repair-triggering candidates produced by the *operational* utility decode - the same candidates the pipeline repairs in Table 5 - with every strategy repairing the identical candidate and boundary on each instance, and separately on the 50 repair-stress cases. On the operational candidates the fixed violated-scope and radius-0 cores fail on 8.5% of repairs (21 of 248), and the expanding strategies resolve all of them, with 21 one-sided discordant pairs this difference is now statistically unambiguous (exact McNemar, Holm-adjusted $p = 6 \times 10^{-6}$). In this ablation each strategy is confined to its own core-construction rule without the pipeline’s escalation, so a “failure” is an infeasible fixed-boundary subproblem on a globally feasible model, the full pipeline never reports such a case as blocked, because it escalates and, if the deadline intervenes, falls back.

Table 5 End-to-end outcomes on the static suite. The first five columns are integer counts summing to N . For the three continuous + repair variants one S5 instance errored before completion and is excluded ($N=249$, complete-case), all other methods completed all 250 instances. Feas. is verified feasibility, ≤ 100 ms is post-hoc budget compliance, and latencies are medians and 95th percentiles. Latencies of GAC + utility decode are omitted because that ablation was executed in the secondary environment, its integer counts are environment-independent.

Method	N	Direct	Local rep.	Full rep.	Fall.	Blk.	Feas. (%)	≤ 100 ms (%)	Med. (ms)	P95 (ms)
Uncoordinated	250	0	0	0	0	250	0.0	0.0	0.0	0.1
Static priority	250	0	0	0	0	250	0.0	0.0	0.0	0.0
GAC + utility decode	250	2	0	0	0	248	0.8	-	-	-
CP-SAT	250	0	0	250	0	0	100.0	100.0	2.8	4.8
GAC + CP-SAT	250	2	0	248	0	0	100.0	100.0	2.6	4.7
ConsistXApp	250	2	248	0	0	0	100.0	100.0	1.8	3.8
Continuous only	250	189	0	0	0	61	75.6	23.6	180.4	365.7
GAC + continuous decode	250	186	0	0	0	64	74.4	44.4	80.6	251.3
Cont. + repair (expl.)	249	185	57	0	0	7	97.2	57.4	79.9	253.3
Cont. + repair (radius)	249	185	57	0	0	7	97.2	57.0	82.6	251.9
Cont. + repair (assum.)	249	185	58	0	0	6	97.6	57.8	81.3	254.4

Table 6 Paired tests on matched static instances, Holm-corrected within each family, complete-case analysis (the errored continuous instance is excluded, and the error is implementation-related, not instance-related). Disc. gives discordant-pair counts (exact McNemar), latency effects are matched-pairs rank-biserial correlations r_{rb} with Hodges-Lehmann median differences (bootstrap 95% CI of the first two rows: $[-0.93, -0.72]$ and $[-0.80, -0.57]$ ms).

Comparison (A vs B)	N	Feas. A/B	Disc.	p_{McN}	Med. A/B (ms)	p_{Wilc} (r_{rb})	HL
CONSISTXAPP vs CP-SAT	250	100/100	0/0	-	1.76/2.76	$< 10^{-33}$ (-0.89)	-0.85
CONSISTXAPP vs GAC+CP-SAT	250	100/100	0/0	-	1.76/2.63	$< 10^{-30}$ (-0.85)	-0.75
Cont.+rep. vs CONSISTXAPP	249	97.2/100	0/7	0.078	79.9/1.75	$< 10^{-40}$ ($+1.0$)	$+94$
Cont.+rep. vs CP-SAT	249	97.2/100	0/7	0.078	79.9/2.76	$< 10^{-40}$ ($+1.0$)	$+93$

Radius-1 fails on 2 cases (not significant) and radius-2 reaches 100%, but only by pre-inflating the core to a median of 16 variables against 11 for explanation-guided expansion ($p \approx 5 \times 10^{-27}$, $r_{rb} = -0.79$), full-scope solving uses the complete model (a median of 20 variables) and is half again as slow as explanation-guided repair (1.78 versus 1.19 ms, $p \approx 8 \times 10^{-34}$, $r_{rb} = -0.89$, Fig. 3). On the repair-stress suite every strategy succeeds - its purpose is core-size contrast - and the tight cores stand out: explanation and assumption expansion hold 8 variables where radius-1 and radius-2 inflate to 16 and 22 and full scope uses all 24. Localization, not raw success, is also what keeps the repair certificate small and readable.

Objective quality. Local repair exactly determines feasibility under its current boundary but does not optimize the full model, so its price must be quantified rather than assumed away. On every scenario instance we compared the kept-request rate of explanation-guided repair of the preferred-request vector against the exact optimum of that objective computed by CP-SAT: repair retains on average 53.8% of preferred requests against 64.6%

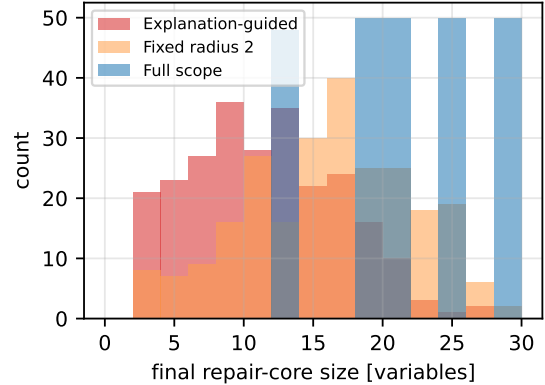


Fig. 3 Final repair-core sizes on the 248 operational repair-triggering candidates.

for the optimizer - a mean gap of 10.8 percentage points (paired Wilcoxon $p \approx 10^{-37}$, the gap is zero on 12.8% of instances and at most 40 points) - at roughly half the latency (median 1.25 versus 2.21 ms). The two capabilities are therefore distinct and should not be conflated: CONSISTXAPP provides *fast feasible repair with an audit trail*,

Table 7 Repair-strategy ablation on the operational candidates (standard suite: the 248 utility-decode failures, stress suite: 50 cases). “Used” counts expansions driven by a wipeout explanation (e), a solver assumption core (a), or the frontier rule (f).

Strategy	Standard (248)					Stress (50)		
	Succ. (%)	Final core	Used (e/a/f)	Rel.	Med. (ms)	Succ. (%)	Final core	Med. (ms)
Violated	91.5	10	0/0/0	0.00	0.97	100	8	0.54
Radius 0	91.5	10	0/0/0	0.00	1.01	100	8	0.51
Radius 1	99.2	14.5	0/0/0	0.00	1.57	100	16	0.65
Radius 2	100.0	16	0/0/0	0.00	1.71	100	22	0.90
Frontier	100.0	11	0/0/21	0.48	1.23	100	8	0.64
Explanation	100.0	11	16/0/8	0.30	1.19	100	8	0.64
Assumption	100.0	10.5	0/21/0	0.15	2.44	100	8	2.50
Full scope	100.0	20	0/0/0	0.00	1.78	100	24	1.37

and complete CP-SAT with an objective provides *globally optimal coordination*, operators who need optimality at every epoch should run the optimizing solver and accept its re-solve cost, while operators bound by the control budget at scale can trade a quantified objective gap for the amortized latency profile of the next subsection.

7.5 Conflict-aware decoding baselines

The utility decode is a deliberately weak candidate generator, and a fair evaluation must ask whether a stronger yet still inexpensive decoder would make repair unnecessary. Table 8 answers this on the 250 static instances. A single greedy pass with immediate constraint checks barely helps (5.6% directly feasible): without backtracking, early choices are invalidated by later binary conflicts. A min-conflicts local search and, especially, a forward-checking sequential search are far stronger, reaching 71.6% and 86.0% direct verified feasibility—two orders of magnitude above the utility decode—which confirms that the headline decode is a strawman and should not by itself be read as evidence for repair. Three facts, however, keep explanation-guided repair central rather than redundant. First, none of the cheap decoders is complete: even forward checking leaves 14% of instances unsolved within its 50 ms budget (its p95 decode latency sits at the budget ceiling), and direct feasibility falls monotonically with instance size, from 100% on the 12-variable S1 to 62% on the 30-variable S5, so the gap the coordination layer must close grows precisely where guarantees

matter. Second, the conflict-aware decoders reach feasibility by discarding preferences: they keep only 55–57% of variables at their most-preferred value against 77.5% for the utility decode, whereas repair minimizes deviation from the decode by construction. Third, none emits a verifiable certificate. Crucially the two are complementary, not competing: explanation-guided repair completes *every* candidate, from all four decoders, to 100% verified feasibility, and a better starting candidate makes repair cheaper—the median repair core falls from 11 variables when repairing the utility decode to 4 when repairing the min-conflicts candidate. On these small cold-start instances a forward-checking decoder is even competitive in median latency; consistent with Section 7.7, the value of the coordination layer is not in beating a decoder on isolated instances but in the feasibility guarantee, the objective control, and the audit trail it adds to whatever candidate a decoder proposes. A production deployment would accordingly pair a conflict-aware decoder with the verified repair completion: the decoder shrinks the repaired fraction, and repair supplies the guarantee and certificate the decoder cannot.

7.6 Incremental propagation

Over the 15,000-epoch dynamic suite, incremental propagation reproduces the fully recomputed filtered domains on every epoch while reducing mean propagation time by 86.3% on restrictive and 95.9% on utility-only transitions, where the warm support state is reused almost in full (Table 9). The gain is smaller for relaxed (50.2%) and mixed (32.0%) transitions because relaxation

Table 8 Conflict-aware decoding baselines on the 250 static ins “Direct” is verifier-accepted feasibility without repair; kept-pref mean fraction of variables left at their most-preferred value; late are the decode-stage median and 95th percentile. Every decoder reaches 100% verified feasibility once its candidate is completed explanation-guided repair (last two columns: instances still require repair, and the median repair-core size on them).

Decoder	Direct (%)	Kept-pref (%)	Med (ms)	P95 (ms)	Rep. needed
Utility decode	0.8	77.5	0.15	0.29	248
Greedy + checks	5.6	71.2	0.05	0.07	236
Min-conflicts	71.6	55.1	0.37	4.50	71
Forward-checking	86.0	57.3	0.19	50.2	35

first restores the affected component to its base domains (0.38 and 0.67 restored values per epoch on average) before reuse can resume - the price of the soundness required by Proposition 4.

7.7 Cold start versus amortized operation

Figure 4 shows the cold-start frontier over twenty seeds per size. Solving isolated near-threshold instances once from scratch, complete CP-SAT stays within the 100 ms budget up to $n = 1000$ (38 ms median, p95 43 ms) whereas the one-shot pipeline approaches it (68 ms median, p95 90 ms), its observable GAC stage alone accounting for roughly 40% of the cost. A deployment that faces isolated, unrelated coordination problems should call the complete solver directly, the architecture cannot be justified there, and we do not attempt to.

The near-real-time RIC, however, does not face isolated problems: consecutive epochs share most of their model, and this is where the architecture earns its cost (Fig. 5, Table 10). The sequential protocol is fully specified: at each epoch the candidate is the previous verified decision, re-verified under the updated relations, utilities and domains are unchanged, only relations move. This is not a trivial re-verification regime - at $n = 1000$, $\delta = 2\%$, the previous decision survives unchanged on only 1.7% of epochs (69% at $n = 100$) - so almost every large-model epoch performs actual repair. With $\delta = 2\%$, verified incremental repair costs a median of 0.05 ms ($n = 100$) to 4.9 ms ($n = 1000$) per epoch, against

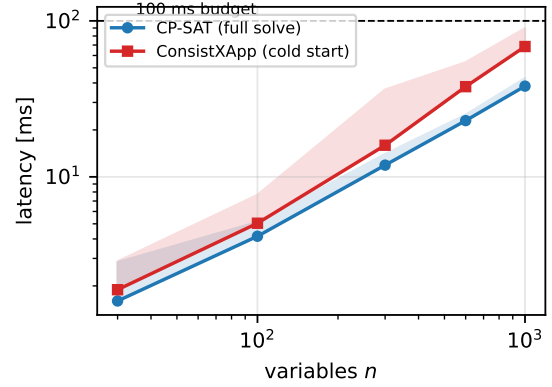


Fig. 4 Cold-start frontier on near-threshold instances (median lines, shaded to p95, twenty seeds per size). One-shot coordination of isolated instances offers no advantage over complete CP-SAT.

2.8–25.6 ms for re-solving from scratch, at identical 100% verified feasibility. Treating the episode as the experimental unit, the per-episode median favours incremental repair in every one of the ten paired episodes at every size and change rate (rank-biserial $r = -1.0$, Wilcoxon $p = 0.002$, the smallest value attainable with ten pairs, Hodges–Lehmann per-epoch difference -20.3 ms at $n = 1000$). Warm-starting the re-solve with solution hints changes nothing. The instrumented decomposition explains why, and corrects a tempting oversimplification: at $n = 1000$ the re-solve spends a median 6.9 ms building the model and 18.4 ms in search, so in our Python/OR-Tools implementation *both* components scale with the model and neither depends on the change fraction - hints reduce neither model handling nor the full-model propagation that dominates this search on feasibility instances - whereas incremental repair scales with the number of violated relations.

The trade-off appears in the tail, and it is sharper than the medians suggest. At $n = 1000$, $\delta = 2\%$, incremental repair reaches re-solve parity at the 95th percentile (30.6 versus 31.2 ms) and crosses over beyond it (p99 84 versus 33 ms, maximum 129 ms), at $\delta = 10\%$ the p95 itself degrades to 60 ms. Deployments with hard per-epoch tail budgets would need to escalate to the full solver when the estimated core grows too large, we have not evaluated a concrete escalation threshold, and designing one is future work. The drift variant

Table 9 Incremental propagation versus full recomputation by transition type (15,000 epochs, mean GAC time per epoch, domains agree on every epoch).

Transition	Count	Recomp. (ms)	Increm. (ms)	Gain (%)	Restored	Agree (%)
Base (full recompute)	150	-	-	-	-	100.0
Restrictive	750	0.102	0.014	86.3	0.00	100.0
Relaxed	750	0.100	0.050	50.2	0.38	100.0
Mixed	12600	0.087	0.059	32.0	0.67	100.0
Utility-only	750	0.099	0.004	95.9	0.00	100.0

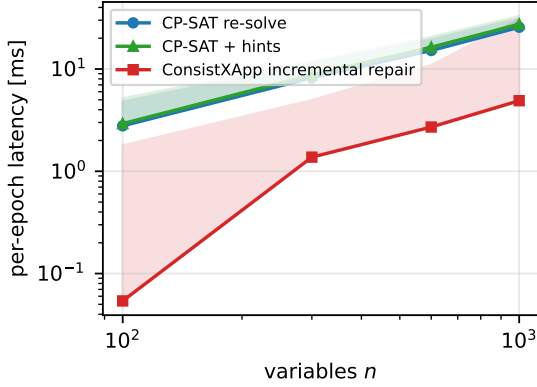


Fig. 5 Amortized per-epoch cost when 2% of the relations change between epochs (median lines, shaded to p95, identical episodes for the three strategies). Re-solving pays in the size of the model, incremental repair in the size of the change.

addresses the concern that a static planted anchor makes the problem artificially easy: re-planting 1% of the variables per epoch invalidates the previous decision on *every* epoch, and incremental repair still wins the median at $n = 1000$ (9.8 versus 28.1 ms) with the same tail caveat (p99 81 ms). Consistently, the temporal-stability sweep on the dynamic sequence shows that raising the churn weight μ from 0 to 1 lowers mean action churn by 11% at a throughput cost below 0.2% with feasibility preserved. Finally, the 15,000-epoch dynamic suite confirms that incremental propagation is exactly correct (100% domain agreement) with per-epoch propagation savings of up to 96%, in absolute terms this saves only fractions of a millisecond, and the large sequential gains above come from avoiding full-model re-solving, not from the propagation savings themselves.

7.8 Network-level effect of verification

Table 11 separates counterfactual KPIs of raw proposals from KPIs of verifier-accepted executed decisions, with bootstrap 95% confidence intervals. No material KPI degradation from enforcing verification was detected in this synthetic model: the intervals of CONSISTXAPP and complete CP-SAT overlap on throughput, energy per bit, and fairness, while uncoordinated execution, whose raw proposals look attractive counterfactually, never produces an executable verified decision on this suite. Feasibility, not KPI optimization, is what separates the methods here, a realistic closed-loop platform is required before any stronger network-level claim.

8 Limitations and Threats to Validity

Every guarantee in this paper is conditional on the registered model. A verified decision satisfies the registered domains, relations, and context, it is not thereby optimal or safe for a physical RAN whose dependencies the model misses, and implicit conflicts in particular are only as trustworthy as the parameter-KPM model, profile, or operator rule that produced their relations. The validation gates establish agreement with an oracle and a second implementation on the tested input distributions, not beyond them, they mutate inputs, not the verifier’s code, so they bound misclassification, not implementation coverage in the mutation-testing sense. Likewise, GAC is a local property - non-empty filtered domains do not certify global feasibility - and a local repair that fails under a fixed boundary says nothing about model infeasibility, which is why escalation and the assumption-core mechanism exist.

Table 10 Amortized per-epoch latency over ten paired episodes of thirty epochs, median (p95, p99 for incremental repair) in ms, all strategies reach 100% verified feasibility on every epoch. Drift rows re-plant 1% of variables per epoch.

δ / n	CP-SAT re-solve	CP-SAT+hints	Incr. repair
2% / 100	2.8 (4.8)	2.9 (5.2)	0.05 (1.8; 4.4)
2% / 300	8.2 (11.4)	8.7 (11.8)	1.4 (5.0; 13.2)
2% / 600	15.2 (19.5)	16.3 (20.7)	2.7 (10.9; 22.7)
2% / 1000	25.6 (31.2)	27.4 (32.8)	4.9 (30.6; 83.7)
10% / 1000	18.3 (27.1)	20.2 (28.6)	4.5 (59.6; 137.8)
drift / 600	17.8 (20.9)	18.9 (22.0)	4.0 (25.8; 69.3)
drift / 1000	28.1 (32.5)	29.8 (34.2)	9.8 (43.8; 81.3)

Table 11 Network KPIs of verifier-accepted executed decisions, mean [bootstrap 95% CI] (energy in $\mu\text{J}/\text{bit}$). The KPI sample of CONSISTXAPP is 249: one verified S5 decision raised a KPI-computation error and is reported as missing rather than imputed. Counterfactual throughput of the raw uncoordinated proposal is 11.7 Mbps with no verified-executed decision ($n=0$).

Method	Verified thr. (Mbps)	Energy	Fairness (Jain)
CP-SAT ($n=250$)	10.84 [10.56, 11.12]	50.8 [49.6, 52.1]	0.443 [0.433, 0.452]
CONSISTXAPP ($n=249$)	10.41 [10.15, 10.68]	52.2 [51.0, 53.5]	0.433 [0.424, 0.443]
Cont.+repair ($n=242$)	10.93 [10.68, 11.18]	50.6 [49.3, 51.9]	0.451 [0.441, 0.461]

The performance picture has explicit boundaries. The cold-start experiment shows no advantage over complete CP-SAT for isolated instances up to a thousand variables, and the observable Python GAC stage carries constants that a production propagator would reduce but not eliminate, the architectural claim is therefore amortized, and it presumes epochs that are genuinely related. The amortized advantage is a median advantage: incremental repair reaches re-solve tail parity at the 95th percentile and crosses over at the 99th ($\delta = 2\%$, $n = 1000$), degrades further at high change rates (p95 of 60 ms at $\delta = 10\%$), and a concrete escalation threshold that would bound the tail has not been evaluated. The sequential evidence rests on ten independent episodes per configuration, analysed at the episode level, larger campaigns would tighten the interval estimates. Local repair is also not an optimizer: it concedes a mean 10.8-point kept-request gap to exact optimization, and deployments requiring per-epoch optimality should prefer the optimizing solver. Explanation quality is likewise not guaranteed - a valid but large explanation can grow the core toward the full problem - and explicit positive tables, convenient for auditability, grow with domain size and arity, compact intensional representations (decision diagrams, automata constraints, dedicated propagators) would preserve

GAC at the cost of more complex explanation extraction. Finally, although the drift variant moves the feasibility anchor every epoch, the sequential generator changes only relations, sequences in which domains, xApp activations, and utilities move jointly remain untested at scale.

The evaluation is simulation-based. The pseudo-physical model produces reproducible KPMs but does not capture E2 delays, scheduling jitter, message loss, or vendor-specific behaviour, and synthetic planted relations may not reproduce every dependency of a commercial multi-vendor RAN. The scenario, dynamic, and temporal campaigns ran on the reference host while the operational-candidate ablation, scale, objective-gap, and validation campaigns ran in a documented secondary environment, comparisons are paired within environments, but absolute latencies differ across them and should be re-measured on one host before deployment conclusions are drawn. Closed-loop validation of at least one dynamic scenario on an emulated O-RAN platform is the natural next step.

9 Conclusion

This paper cast multi-xApp coordination in the near-real-time RIC as a dynamic finite-domain constraint optimization problem and presented

CONSISTXAPP: incremental generalized arc consistency with sound restoration, explanation-guided exact local repair with monotone core expansion, and an independently implemented verifier gating every E2 action. Propagation is provably solution-preserving, repair is conditionally complete, and the verifier is validated by exhaustive, dual-implementation, property-based, and oracle-labelled input-mutation testing with zero divergence across 680,450 checks.

Empirically, CONSISTXAPP attains 100% verified feasibility on five O-RAN conflict scenarios at a median 1.8 ms - a statistically firm but operationally modest millisecond ahead of complete CP-SAT- and on the 248 repairs actually triggered by its pipeline, expanding repair cores succeed significantly more often than fixed cores while staying approximately half the size of full-scope solving. The decisive property is amortization: when 2% of the model changes between epochs, repairing the previous verified decision costs a median of 0.05–4.9 ms per epoch across 100–1,000 variables where either from-scratch re-solve, hinted or not, costs 2.8–25.6 ms, an advantage that held in every paired episode and persists when the feasibility anchor drifts. Three boundaries are reported with equal clarity: isolated cold-start instances are better served by the complete solver, the amortized advantage concerns the median, with tail parity at the 95th percentile and crossover beyond it, and local repair concedes a quantified objective gap (10.8 points of kept-request rate on average) to exact optimization. A continuous-relaxation stage that we evaluated degrades both latency and feasibility and was excluded from the pipeline.

Future work will address compact relation representations with auditable explanations, explanation minimization, tail-bounding escalation policies, uncertainty-aware conflict relations, distributed propagation across multiple RICs, and closed-loop validation on an emulated O-RAN platform.

Statements and Declarations

Funding

No funding was received for conducting this study.

Competing interests

The authors declare that they have no competing financial or non-financial interests that are directly or indirectly related to this work.

Author contributions

Lillia Ouali: Conceptualization, Validation, Supervision. Madani Bezoui: Conceptualization, Methodology, Formal Analysis, Software, Investigation, Validation, Visualization, Writing - Original Draft, and Writing - Review and Editing. Kamal Amroun: Validation, Writing - Review and Editing. Ahcene Bounceur: Validation, Supervision. All authors reviewed and approved the final manuscript.

Data and code availability

The code, generated instances, raw logs, explanation traces, verifier-validation reports, the primary and secondary reproducibility manifests (hardware, software versions, seed hierarchy, and command lines), and the scripts reproducing every table, figure, and statistical test are provided as a review artifact and will be deposited in a public archival repository with a persistent identifier upon acceptance.

Ethics approval, consent to participate, consent for publication

Not applicable.

Use of generative artificial intelligence

Generative language models were used to assist with the organization and language drafting of the manuscript and with the review of the analysis scripts. The authors reviewed, corrected, and validated the resulting text and code and assume full responsibility for the scientific content, mathematical statements, references, experimental protocol, numerical results, and final submitted version.

References

- [1] Polese, M., Bonati, L., D’Oro, S., Basagni, S., Melodia, T.: Understanding O-RAN:

- Architecture, interfaces, algorithms, security, and research challenges. *IEEE Communications Surveys & Tutorials* **25**(2), 1376–1411 (2023) <https://doi.org/10.1109/COMST.2023.3239220>
- [2] O-RAN Alliance: O-RAN Conflict Mitigation, Version 1.00. Technical report, Near-Real-Time RIC and E2 Interface Work Group. Specification portal accessed 14 July 2024 (2024). <https://specifications.o-ran.org/>
- [3] Adamczyk, C., Kliks, A.: Conflict mitigation framework and conflict detection in O-RAN Near-RT RIC. *IEEE Communications Magazine* **61**(12), 199–205 (2023) <https://doi.org/10.1109/MCOM.018.2200752>
- [4] Wadud, A., Golpayegani, F., Afraz, N.: QACM: QoS-aware xApp conflict mitigation in Open RAN. *IEEE Transactions on Green Communications and Networking* **8**(3), 978–993 (2024) <https://doi.org/10.1109/TGCN.2024.3431945>
- [5] Polese, M., Bonati, L., D’Oro, S., Basagni, S., Melodia, T.: ColO-RAN: Developing machine learning-based xapps for open ran closed-loop control on programmable experimental platforms. *IEEE Transactions on Mobile Computing* (2023) <https://doi.org/10.1109/TMC.2022.3188013>
- [6] Bonati, L., Polese, M., D’Oro, S., Basagni, S., Melodia, T.: OpenRAN Gym: Ai/ml development, data collection, and testing for O-RAN on pawr platforms. *Computer Networks* **220**, 109502 (2023) <https://doi.org/10.1016/j.comnet.2022.109502>
- [7] Schmidt, R., Irazabal, M., Nikaein, N.: FlexRIC: An sdk for next-generation sd-rans. In: *Proceedings of the 17th International Conference on Emerging Networking Experiments and Technologies*, pp. 411–425. Association for Computing Machinery, New York, NY, USA (2021). <https://doi.org/10.1145/3485983.3494870>
- [8] Wadud, A., Golpayegani, F., Afraz, N.: Towards efficient conflict mitigation in the converged 6G Open RAN control plane. *Annals of Telecommunications* (2024) <https://doi.org/10.1007/s12243-024-01036-2>
- [9] Wadud, A., Golpayegani, F., Afraz, N.: Conflict management in the Near-RT-RIC of Open RAN: A Game Theoretic approach. In: *Proc. IEEE International Conferences on Internet of Things (iThings) and Green Computing and Communications (GreenCom) and Cyber, Physical and Social Computing (CPSCom)* (2023)
- [10] Prever, P., D’Oro, S., Bonati, L., Polese, M., Tsampazi, M., Lehmann, H., Melodia, T.: PACIFISTA: Conflict evaluation and management in open ran. *IEEE Transactions on Mobile Computing* (2025) <https://doi.org/10.1109/TMC.2025.3570632>
- [11] Giannopoulos, A.E., *et al.*: COMIX: Generalized conflict management in O-RAN xapps—architecture, workflow, and a power control case. *IEEE Access* (2025) <https://doi.org/10.1109/ACCESS.2025.3585774>
- [12] Wadud, A., Golpayegani, F., Afraz, N.: xApp-level conflict mitigation in O-RAN, a mobility driven energy saving case. In: *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)* (2025)
- [13] Varshney, N., *et al.*: Explainable artificial intelligence for conflict management: XAI4C for conflict detection and mitigation in O-RAN Near-RT RIC. *IEEE Vehicular Technology Magazine* (2025) <https://doi.org/10.1109/MVT.2025.3625693>
- [14] Yap, R.H.C., Xia, W., Wang, R.: Generalized arc consistency algorithms for table constraints: A summary of algorithmic ideas. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, pp. 13590–13597 (2020). <https://doi.org/10.1609/aaai.v34i09.7086>
- [15] Lecoutre, C.: STR2: Optimized simple tabular reduction for table constraints. *Constraints* **16**(4), 341–371 (2011) <https://doi.org/10.1007/s10601-011-9107-6>

- [16] Bessière, C.: Arc-consistency in dynamic constraint satisfaction problems. In: Proceedings of the Ninth National Conference on Artificial Intelligence, pp. 221–226 (1991)
- [17] Dev Gupta, S., Genç, B., O’Sullivan, B.: Explanation in constraint satisfaction: A survey. In: Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence (2021). <https://doi.org/10.24963/ijcai.2021/601>
- [18] Perron, L., Didier, F., Gay, S.: The CP-SAT-LP solver. In: 29th International Conference on Principles and Practice of Constraint Programming. Leibniz International Proceedings in Informatics (2023). <https://doi.org/10.4230/LIPIcs.CP.2023.3> . Using OR-Tools v9.11
- [19] Perron, L., Didier, F.: CP-SAT Solver. Google OR-Tools documentation. Accessed 14 July 2024 (2023). https://developers.google.com/optimization/cp/cp_solver
- [20] Yedidia, J.S., Freeman, W.T., Weiss, Y.: Understanding belief propagation and its generalizations. In: Exploring Artificial Intelligence in the New Millennium, pp. 239–269. Morgan Kaufmann, San Francisco, CA, USA (2003)